CODE REFERENCE · PARABANK AUTOMATION FRAMEWORK

ParaBank Automation Framework

Complete Source Code — All 18 Files

PROJECT	ParabankAutomationFramework
LANGUAGE	Java 17
FRAMEWORK	Selenium 4.34.0 · TestNG 7.11.0
REPORTING	ExtentReports 5.1.2 · Log4j2 2.25.1
BUILD	Maven · Surefire 3.5.3
LAYERS	Base · Pages · Tests · Utilities · Listeners
DATE	June 15, 2026

LAYER LEGEND

Base · Listener · Page · Test · Utility

18 source files documented

Table of Contents

Base Layer

Browser lifecycle management and WebDriver initialisation

BaseFile.java base Base 82 lines

```java
package base;

import java.time.Duration;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Optional;
import org.testng.annotations.Parameters;

import utilities.ConfigReader;

public class BaseFile {

private static final Logger logger =
LogManager.getLogger(BaseFile.class);

protected static WebDriver driver;

public static WebDriver getDriver() {
return driver;
}

 @BeforeMethod
 @Parameters("browser")
public void setup(@Optional("") String browser) {

if (browser == null || browser.trim().isEmpty()) {
browser = ConfigReader.getProperty("browser");
}

logger.info("Launching Browser: {}", browser);

switch (browser.toLowerCase()) {

case "chrome":
driver = new ChromeDriver();
break;

case "edge":
driver = new EdgeDriver();
```

```
47    break;
48
49    case "firefox":
50    driver = new FirefoxDriver();
51    break;
52
53    default:
54    throw new IllegalArgumentException("Invalid browser: " + browser);
55    }
56
57    driver.manage().window().maximize();
58
59    driver.manage().timeouts().implicitlyWait(
60    Duration.ofSeconds(
61    Integer.parseInt(
62    ConfigReader.getProperty("implicitWait"))));
63
64    String url = ConfigReader.getProperty("url");
65
66    logger.info("Opening URL: {}", url);
67
68    driver.get(url);
69    }
70
71     @AfterMethod(alwaysRun = true)
72    public void tearDown() {
73
74    if (driver != null) {
75
76    logger.info("Closing Browser");
77
78    driver.quit();
79    driver = null;
80    }
81    }
82    }
```

Listener Layer

TestNG ITestListener hooks for reporting and screenshots

TestListener.java listeners Listener 66 lines

```
1    package listeners;
2
3    import org.testng.ITestContext;
4    import org.testng.ITestListener;
5    import org.testng.ITestResult;
6
7    import com.aventstack.extentreports.ExtentReports;
8    import com.aventstack.extentreports.ExtentTest;
```

```
9
10  import base.BaseFile;
11  import utilities.ExtentManager;
12  import utilities.ScreenshotUtils;
13
14  public class TestListener
15  implements ITestListener {
16
17  ExtentReports extent =
18  ExtentManager.getReport();
19
20  ExtentTest test;
21
22   @Override
23  public void onTestStart(
24  ITestResult result) {
25
26  test =
27  extent.createTest(
28  result.getName());
29  }
30
31   @Override
32  public void onTestSuccess(
33  ITestResult result) {
34
35  test.pass("Test Passed");
36  }
37
38   @Override
39  public void onTestFailure(
40  ITestResult result) {
41
42  test.fail(result.getThrowable());
43
44  String screenshotPath =
45  ScreenshotUtils.captureScreenshot(
46  BaseFile.getDriver(),
47  result.getName());
48
49  try {
50
51  test.addScreenCaptureFromPath(
52  screenshotPath);
53
54  } catch (Exception e) {
55
56  e.printStackTrace();
57  }
58  }
59
60   @Override
61  public void onFinish(
```

```
ITestContext context) {

extent.flush();
}
}
```

Page Object Layer

UI element locators and page-level action methods

LoginPage.java pages Page 43 lines

```
package pages;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {
private static final Logger logger =
LogManager.getLogger(LoginPage.class);

WebDriver driver;

public LoginPage(WebDriver driver) {

this.driver = driver;
}

By username = By.name("username");

By password = By.name("password");

By loginBtn = By.xpath("//input[@value='Log In']");

By logoutLink = By.linkText("Log Out");

public void enterUsername(String user) {
logger.info("Entering Username");
driver.findElement(username).clear();
driver.findElement(username).sendKeys(user);
}

public void enterPassword(String pass) {
logger.info("Entering Passwprd");
driver.findElement(password).clear();
driver.findElement(password).sendKeys(pass);
}

public void clickLogin() {
```

```
    logger.info("Click Login Button");
    driver.findElement(loginBtn).click();
    }
    }
```

RegistrationPage.java

pages · Page · 54 lines

```
    package pages;

    import org.apache.logging.log4j.LogManager;
    import org.apache.logging.log4j.Logger;
    import org.openqa.selenium.By;
    import org.openqa.selenium.WebDriver;

    public class RegistrationPage {
    private static final Logger logger =
    LogManager.getLogger(RegistrationPage.class);
    WebDriver driver;
    public RegistrationPage(WebDriver driver)
    {
    this.driver=driver;
    }
    By registerLink=By.linkText("Register");
    By firstName=By.id("customer.firstName");
    By lastName=By.id("customer.lastName");
    By address=By.id("customer.address.street");
    By city=By.id("customer.address.city");
    By state=By.id("customer.address.state");
    By zipCode=By.id("customer.address.zipCode");
    By phoneNumber =By.id("customer.phoneNumber");
    By ssn=By.id("customer.ssn");
    By username=By.name("customer.username");
    By password=By.name("customer.password");
    By confirmPassword=By.id("repeatedPassword");
    By registerBtn = By.xpath("//input[@value='Register']");
    public void clickRegisterLink() {
    logger.info("Clicking Register Button");
    driver.findElement(registerLink).click();
    }
    public void registerUser(String fName, String lName,
    String addr, String cityName,
    String stateName, String zip,
    String phoneNum, String ssnNum,
    String user, String pass) {
    logger.info("Entering Registration Details");
    driver.findElement(firstName).sendKeys(fName);
    driver.findElement(lastName).sendKeys(lName);
    driver.findElement(address).sendKeys(addr);
    driver.findElement(city).sendKeys(cityName);
    driver.findElement(state).sendKeys(stateName);
    driver.findElement(zipCode).sendKeys(zip);
```

```
45    driver.findElement(phoneNumber).sendKeys(phoneNum);
46    driver.findElement(ssn).sendKeys(ssnNum);
47    driver.findElement(username).sendKeys(user);
48    driver.findElement(password).sendKeys(pass);
49    driver.findElement(confirmPassword).sendKeys(pass);
50
51    driver.findElement(registerBtn).click();
52    }
53    }
54
```

AccountViewPage.java

pages | Page | 22 lines

```
1     package pages;
2
3     import org.openqa.selenium.By;
4     import org.openqa.selenium.WebDriver;
5
6     public class AccountViewPage {
7
8     WebDriver driver;
9
10    public AccountViewPage(WebDriver driver) {
11    this.driver = driver;
12    }
13
14    By accountsOverviewHeading =
15    By.xpath("//h1[contains(text(),'Accounts Overview')]");
16
17    public boolean isviewDisplayed() {
18
19    return driver.findElement(accountsOverviewHeading)
20    .isDisplayed();
21    }
22    }
```

TransferFundsPage.java

pages | Page | 131 lines

```
1     package pages;
2
3     import java.time.Duration;
4
5     import org.apache.logging.log4j.LogManager;
6     import org.apache.logging.log4j.Logger;
7     import org.openqa.selenium.By;
8     import org.openqa.selenium.JavascriptExecutor;
9     import org.openqa.selenium.WebDriver;
10    import org.openqa.selenium.WebElement;
11    import org.openqa.selenium.support.ui.ExpectedConditions;
12    import org.openqa.selenium.support.ui.Select;
```

```
13  import org.openqa.selenium.support.ui.WebDriverWait;
14
15  public class TransferFundsPage {
16
17  WebDriver driver;
18  WebDriverWait wait;
19  private static final Logger logger =
20  LogManager.getLogger(TransferFundsPage.class);
21  public TransferFundsPage(WebDriver driver) {
22
23  this.driver = driver;
24  wait = new WebDriverWait(driver, Duration.ofSeconds(10));
25  }
26
27  By transferFundsLink = By.linkText("Transfer Funds");
28
29  By amount = By.id("amount");
30
31  By fromAccount = By.id("fromAccountId");
32
33  By toAccount = By.id("toAccountId");
34
35  By transferButton =
36  By.xpath("//input[@value='Transfer']");
37
38  By successMessage =
39  By.xpath("//h1[contains(text(),'Transfer Complete')]");
40
41
42  public void enterAmount(String amt) {
43  logger.info("Entering Transfer Amount : " + amt);
44
45  wait.until(ExpectedConditions.visibilityOfElementLocated(
46  amount));
47
48  driver.findElement(amount).clear();
49
50  driver.findElement(amount).sendKeys(amt);
51  }
52  public void openTransferFunds() {
53  logger.info("Opening Transfer Funds Page");
54
55  utilities.WaitUtils.waitForElement(
56  driver,
57  By.linkText("Transfer Funds"));
58
59  driver.findElement(
60  By.linkText("Transfer Funds"))
61  .click();
62  }
63
64  public void selectAccounts() {
65
```

```
66    Select from =
67    new Select(driver.findElement(fromAccount));
68
69    Select to =
70    new Select(driver.findElement(toAccount));
71
72    String fromAcc =
73    from.getFirstSelectedOption().getText();
74
75    for (WebElement option : to.getOptions()) {
76
77    String toAcc = option.getText();
78
79    if (!toAcc.equals(fromAcc)) {
80
81    to.selectByVisibleText(toAcc);
82
83    break;
84    }
85    }
86    }
87    public void selectFromAccount() {
88
89    Select from =
90    new Select(driver.findElement(fromAccount));
91
92    from.selectByIndex(0);
93    }
94
95    public void selectToAccount() {
96
97    Select to =
98    new Select(driver.findElement(toAccount));
99
100   to.selectByIndex(0);
101   }
102
103   public void clickTransfer() {
104   logger.info("Clicking Transfer Button");
105
106   WebElement button =
107   driver.findElement(transferButton);
108
109   JavascriptExecutor js =
110   (JavascriptExecutor) driver;
111
112   js.executeScript(
113   "arguments[0].click();",
114   button);
115   }
116
117   public String getSuccessMessage() {
118
```

```
119     wait.until(ExpectedConditions.visibilityOfElementLocated(
120     successMessage));
121
122     return driver.findElement(successMessage)
123     .getText();
124     }
125
126     public String getPageText() {
127
128     return driver.findElement(By.tagName("body"))
129     .getText();
130     }
131     }
```

BillPage.java

pages | Page | 113 lines

```
1     package pages;
2
3     import java.time.Duration;
4
5     import org.apache.logging.log4j.LogManager;
6     import org.apache.logging.log4j.Logger;
7     import org.openqa.selenium.By;
8     import org.openqa.selenium.WebDriver;
9     import org.openqa.selenium.support.ui.ExpectedConditions;
10    import org.openqa.selenium.support.ui.Select;
11    import org.openqa.selenium.support.ui.WebDriverWait;
12
13    public class BillPage {
14
15    private static final Logger logger =
16    LogManager.getLogger(BillPage.class);
17    WebDriver driver;
18
19    public BillPage(WebDriver driver) {
20
21    this.driver = driver;
22    }
23    By payeeName =
24    By.name("payee.name");
25
26    By address =
27    By.name("payee.address.street");
28
29    By city =
30    By.name("payee.address.city");
31
32    By state =
33    By.name("payee.address.state");
34
35    By zipCode =
```

```
    By.name("payee.address.zipCode");

    By phone =
    By.name("payee.phoneNumber");

    By account =
    By.name("payee.accountNumber");

    By verifyAccount =
    By.name("verifyAccount");

    By amount =
    By.name("amount");

    By fromAccount =
    By.name("fromAccountId");

    By sendPayment =
    By.cssSelector(
    "input[value='Send Payment']");
    public void enterPayeeInformation(
    String payee,
    String addr,
    String cityName,
    String stateName,
    String zipcode,
    String phoneNo,
    String accountNo,
    String verifyAccountNo,
    String amt) {
    logger.info("Entering Payee Information");
    driver.findElement(payeeName)
    .sendKeys(payee);

    driver.findElement(address)
    .sendKeys(addr);

    driver.findElement(city)
    .sendKeys(cityName);

    driver.findElement(state)
    .sendKeys(stateName);

    driver.findElement(zipCode)
    .sendKeys(zipcode);

    driver.findElement(phone)
    .sendKeys(phoneNo);

    driver.findElement(account)
    .sendKeys(accountNo);

```

```
88    driver.findElement(verifyAccount)
89    .sendKeys(verifyAccountNo);
90
91    driver.findElement(amount)
92    .sendKeys(amt);
93
94    Select dropdown =
95    new Select(
96    driver.findElement(fromAccount));
97
98    dropdown.selectByIndex(0);
99    }
100   public void openBillPayPage() {
101   logger.info("Opening Bill Pay Page");
102
103   driver.findElement(
104   By.linkText("Bill Pay"))
105   .click();
106   }
107
108   public void clickSendPayment() {
109   logger.info("Sending Bill Payment");
110   driver.findElement(sendPayment)
111   .click();
112   }
113   }
```

Test Case Layer

TestNG @Test methods with data-driven execution

LoginTest.java testcases Test 28 lines

```
1     package testcases;
2
3     import org.testng.Assert;
4     import org.testng.annotations.Test;
5
6     import base.BaseFile;
7     import pages.LoginPage;
8     import utilities.DataProviders;
9
10    public class LoginTest extends BaseFile {
11
12     @Test(priority=2,
13    dataProvider = "loginData",
14    dataProviderClass = DataProviders.class
15    )
16    public void verifyLogin(String username,
17    String password) {
18
```

```
    LoginPage login =
    new LoginPage(BaseFile.getDriver());

    login.enterUsername(username);

    login.enterPassword(password);

    login.clickLogin();
    }
    }
```

RegistrationTest.java

testcases | Test | 43 lines

```
    package testcases;

    import org.testng.annotations.Test;

    import base.BaseFile;
    import pages.RegistrationPage;
    import utilities.DataProviders;

    public class RegistrationTest extends BaseFile {

     @Test(priority=1,dataProvider = "registrationData",
    dataProviderClass = DataProviders.class)

    public void verifyRegistration(
    String firstName,
    String lastName,
    String address,
    String city,
    String state,
    String zip,
    String phone,
    String ssn,
    String username,
    String password) {

    RegistrationPage register =
    new RegistrationPage(BaseFile.getDriver());

    register.clickRegisterLink();

    register.registerUser(
    firstName,
    lastName,
    address,
    city,
    state,
    zip,
    phone,
```

```
39    ssn,
40    username,
41    password);
42    }
43    }
```

AccountViewTest.java

testcases · Test · 33 lines

```
1     package testcases;
2
3     import org.testng.Assert;
4     import org.testng.annotations.Test;
5
6     import base.BaseFile;
7     import pages.AccountViewPage;
8     import pages.LoginPage;
9     import utilities.DataProviders;
10
11    public class AccountViewTest extends BaseFile {
12
13     @Test(priority=3,dataProvider = "loginData",
14    dataProviderClass = DataProviders.class)
15
16    public void verifyAccountOverview(
17    String username,
18    String password) {
19
20    LoginPage login =
21    new LoginPage(BaseFile.getDriver());
22
23    login.enterUsername(username);
24    login.enterPassword(password);
25    login.clickLogin();
26
27    AccountViewPage accountPage =
28    new AccountViewPage(BaseFile.getDriver());
29
30    Assert.assertTrue(
31    accountPage.isviewDisplayed());
32    }
33    }
```

TransferFundsTest.java

testcases · Test · 32 lines

```
1     package testcases;
2
3     import org.testng.Assert;
4     import org.testng.annotations.Test;
5
6     import base.BaseFile;
7     import pages.LoginPage;
```

```java
import pages.TransferFundsPage;
import utilities.DataProviders;

public class TransferFundsTest extends BaseFile {

 @Test(
priority = 5,
dataProvider = "transferData",
dataProviderClass = DataProviders.class
)
public void verifyTransferFunds(
String username,
String password,
String amount) throws InterruptedException {

LoginPage login =
new LoginPage(BaseFile.getDriver());

login.enterUsername(username);

login.enterPassword(password);

login.clickLogin();
}
}
```

BillTest.java

testcases | Test | 79 lines

```java
package testcases;

import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.Test;

import base.BaseFile;
import pages.BillPage;
import pages.LoginPage;
import utilities.DataProviders;
import utilities.ExcelUtils;

public class BillTest extends BaseFile {

 @Test(priority = 6,
dataProvider = "billPayData",
dataProviderClass = DataProviders.class)
public void verifyBillPayment(
String payeeName,
String address,
String city,
String state,
String zipCode,
```

```
    String phone,
    String accountNo,
    String amount) throws Exception {

    System.out.println("===== DATA FROM EXCEL =====");
    System.out.println("Payee Name : " + payeeName);
    System.out.println("Address : " + address);
    System.out.println("City : " + city);
    System.out.println("State : " + state);
    System.out.println("Zip : " + zipCode);
    System.out.println("Phone : " + phone);
    System.out.println("Account No : " + accountNo);
    System.out.println("Amount : " + amount);
    System.out.println("============================");

    LoginPage login = new LoginPage(BaseFile.getDriver());

    login.enterUsername(
    ExcelUtils.getCellData("Login", 1, 0));

    login.enterPassword(
    ExcelUtils.getCellData("Login", 1, 1));

    login.clickLogin();

    BillPage billPay =
    new BillPage(BaseFile.getDriver());

    billPay.openBillPayPage();

    billPay.enterPayeeInformation(
    payeeName,
    address,
    city,
    state,
    zipCode,
    phone,
    accountNo,
    accountNo,
    amount);

    billPay.clickSendPayment();

    Thread.sleep(5000);

    String pageText =
    BaseFile.getDriver().findElement(By.tagName("body"))
    .getText();

    System.out.println(pageText);

    Assert.assertTrue(
```

```
76      pageText.contains("Bill Payment Complete"),
77      "Bill payment failed");
78      }
79      }
```

Utility Layer

Shared helpers — Excel, config, reporting, waits, screenshots

ConfigReader.java

utilities | Utility | 31 lines

```
1       package utilities;
2
3       import java.io.FileInputStream;
4       import java.io.IOException;
5       import java.util.Properties;
6
7       public class ConfigReader {
8
9       private static Properties prop;
10
11      static {
12
13      try {
14
15      FileInputStream fis =
16      new FileInputStream("src/test/resources/config.properties");
17
18      prop = new Properties();
19      prop.load(fis);
20
21      } catch (IOException e) {
22
23      e.printStackTrace();
24      }
25      }
26
27      public static String getProperty(String key) {
28
29      return prop.getProperty(key);
30      }
31      }
```

DataProviders.java

utilities | Utility | 69 lines

```
1       package utilities;
2
3       import org.testng.annotations.DataProvider;
4
5       public class DataProviders {
```

```
6
7       @DataProvider(name = "registrationData")
8      public Object[][] getRegistrationData() {
9
10     return new Object[][] {
11
12     {
13     ExcelUtils.getCellData("Registration",1,0),
14     ExcelUtils.getCellData("Registration",1,1),
15     ExcelUtils.getCellData("Registration",1,2),
16     ExcelUtils.getCellData("Registration",1,3),
17     ExcelUtils.getCellData("Registration",1,4),
18     ExcelUtils.getCellData("Registration",1,5),
19     ExcelUtils.getCellData("Registration",1,6),
20     ExcelUtils.getCellData("Registration",1,7),
21     ExcelUtils.getCellData("Registration",1,8),
22     ExcelUtils.getCellData("Registration",1,9)
23     }
24     };
25     }
26
27      @DataProvider(name = "loginData")
28     public Object[][] getLoginData() {
29
30     return new Object[][] {
31
32     {
33     ExcelUtils.getCellData("Login",1,0),
34     ExcelUtils.getCellData("Login",1,1)
35     }
36     };
37     }
38
39      @DataProvider(name = "transferData")
40     public Object[][] getTransferData() {
41
42     return new Object[][] {
43
44     {
45     ExcelUtils.getCellData("Transfer",1,0),
46     ExcelUtils.getCellData("Transfer",1,1),
47     ExcelUtils.getCellData("Transfer",1,2)
48     }
49     };
50     }
51
52      @DataProvider(name = "billPayData")
53     public Object[][] getBillPayData() {
54
55     return new Object[][] {
56
57     {
```

```
58    ExcelUtils.getCellData("BillPay",1,0),
59    ExcelUtils.getCellData("BillPay",1,1),
60    ExcelUtils.getCellData("BillPay",1,2),
61    ExcelUtils.getCellData("BillPay",1,3),
62    ExcelUtils.getCellData("BillPay",1,4),
63    ExcelUtils.getCellData("BillPay",1,5),
64    ExcelUtils.getCellData("BillPay",1,6),
65    ExcelUtils.getCellData("BillPay",1,7)
66    }
67    };
68    }
69    }
```

ExcelUtils.java

utilities | Utility | 59 lines

```
1     package utilities;
2
3     import java.io.FileInputStream;
4
5     import org.apache.poi.ss.usermodel.DataFormatter;
6     import org.apache.poi.ss.usermodel.Sheet;
7     import org.apache.poi.ss.usermodel.Workbook;
8     import org.apache.poi.ss.usermodel.WorkbookFactory;
9
10    public class ExcelUtils {
11
12    private static final String FILE_PATH =
13    "src/test/resources/testdata/ParabankData.xlsx";
14
15    public static String getCellData(String sheetName,
16    int rowNum,
17    int colNum) {
18
19    try {
20
21    FileInputStream fis =
22    new FileInputStream(FILE_PATH);
23
24    Workbook workbook =
25    WorkbookFactory.create(fis);
26
27    Sheet sheet =
28    workbook.getSheet(sheetName);
29
30    if(sheet == null) {
31
32    System.out.println(
33    "Sheet NOT FOUND : " + sheetName);
34
35    workbook.close();
36
37    return "";
```

```
    }

    DataFormatter formatter =
    new DataFormatter();

    String value =
    formatter.formatCellValue(
    sheet.getRow(rowNum)
    .getCell(colNum));

    workbook.close();

    return value;

    } catch (Exception e) {

    e.printStackTrace();
    }

    return "";
    }
    }
```

ExtentManager.java

utilities | Utility | 32 lines

```
    package utilities;

    import com.aventstack.extentreports.ExtentReports;
    import com.aventstack.extentreports.reporter.ExtentSparkReporter;

    public class ExtentManager {

    private static ExtentReports extent;

    public static ExtentReports getReport() {

    if (extent == null) {

    ExtentSparkReporter spark =
    new ExtentSparkReporter(
    "reports/ExtentReport.html");

    spark.config()
    .setReportName("ParaBank Automation Report");

    extent = new ExtentReports();

    extent.attachReporter(spark);

    extent.setSystemInfo(
    "Tester",
    "Susovan Maji");
```

```
    }

    return extent;
    }
    }
```

ScreenshotUtils.java

utilities | Utility | 48 lines

```
    package utilities;

    import java.io.File;
    import java.io.IOException;
    import java.text.SimpleDateFormat;
    import java.util.Date;

    import org.apache.commons.io.FileUtils;
    import org.openqa.selenium.OutputType;
    import org.openqa.selenium.TakesScreenshot;
    import org.openqa.selenium.WebDriver;

    public class ScreenshotUtils {

    public static String captureScreenshot(
    WebDriver driver,
    String testName) {

    String timestamp =
    new SimpleDateFormat("yyyyMMdd_HHmmss")
    .format(new Date());

    String path =
    "screenshots/"
    + testName
    + "_"
    + timestamp
    + ".png";

    File source =
    ((TakesScreenshot) driver)
    .getScreenshotAs(
    OutputType.FILE);

    try {

    FileUtils.copyFile(
    source,
    new File(path));

    } catch (IOException e) {

    e.printStackTrace();
```

```
    }

    return path;
    }
    }
```

WaitUtils.java

utilities | Utility | 25 lines

```
package utilities;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitUtils {

public static void waitForElement(
WebDriver driver,
By locator) {

WebDriverWait wait =
new WebDriverWait(
driver,
Duration.ofSeconds(10));

wait.until(
ExpectedConditions.visibilityOfElementLocated(
locator));
}
}
```

Config & Build Files

Properties, Jenkins pipeline, and Maven POM

config.properties

src/test/resources | Config | 4 lines

```
browser=chrome
url=https://parabank.parasoft.com/parabank/index.htm
implicitWait=10

```

Jenkinsfile

root | Config | 50 lines

```
pipeline {


agent any

tools {

maven 'Maven'
}

stages {

stage('Checkout') {

steps {

checkout scm
}
}

stage('Build & Test') {

steps {

bat 'mvn clean test'
}
}
}

post {

always {

archiveArtifacts artifacts: 'test-output/**', allowEmptyArchive: true
}

success {

```

```
    echo 'Automation Suite Passed'
    }

    failure {

    echo 'Automation Suite Failed'
    }
    }


    }

```

pom.xml

root · Config · 111 lines

```
    <project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation=
    https://maven.apache.org/xsd/maven-4.0.0.xsd

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.susovan</groupId>
    <artifactId>ParabankAutomationFramework</artifactId>
    <version>1.0</version>

    <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

    <selenium.version>4.34.0</selenium.version>
    <testng.version>7.11.0</testng.version>
    <cucumber.version>7.23.0</cucumber.version>
    </properties>

    <dependencies>

    <!-- Selenium -->
    <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>${selenium.version}</version>
    </dependency>

    <!-- TestNG -->
    <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
    <scope>test</scope>
```

```
    </dependency>

    <!-- WebDriverManager -->
    <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>6.3.2</version>
    </dependency>

    <!-- Apache POI -->
    <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.4.1</version>
    </dependency>

    <!-- Extent Reports -->
    <dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.2</version>
    </dependency>

    <!-- Log4j -->
    <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>2.25.1</version>
    </dependency>

    <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.25.1</version>
    </dependency>

    <!-- Screenshot Support -->
    <dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.20.0</version>
    </dependency>

    </dependencies>

    <build>
    <plugins>

    <!-- Compiler Plugin -->
    <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
```

```
    <version>3.14.0</version>
    <configuration>
    <source>17</source>
    <target>17</target>
    </configuration>
    </plugin>

    <!-- TestNG Runner -->
    <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.3</version>
    <configuration>
    <suiteXmlFiles>
    <suiteXmlFile>testng.xml</suiteXmlFile>
    </suiteXmlFiles>
    </configuration>
    </plugin>

    </plugins>
    </build>

    </project>
```